SLIDE 10

MỘT LƯỢT SUY LUẬN MRI

Từ ảnh đầu vào đến kết quả ARM + FPGA trên PYNQ-Z2

Bài giảng thực hành • Diễn giải dữ liệu • Luyện thuyết minh

Vì sao cần học riêng một lượt suy luận?

– Giúp người học theo dõi chính xác **dữ liệu đang nằm ở đâu** và được xử lý bằng gì.

– Liên hệ các khái niệm đã học: **INT8, Conv2D, DMA, Overlay và TFLite**.

– Tập giảng từ một ví dụ thực nghiệm, không chỉ đọc lại sơ đồ và bảng kết quả.

– Phân biệt **kết quả một ảnh** với **đánh giá toàn bộ 995 ảnh Test**.

Ví dụ xuyên suốt bài học

Một ảnh MRI thật được dùng để chạy chương trình Lab 05.

Tên ảnh kiểm thử

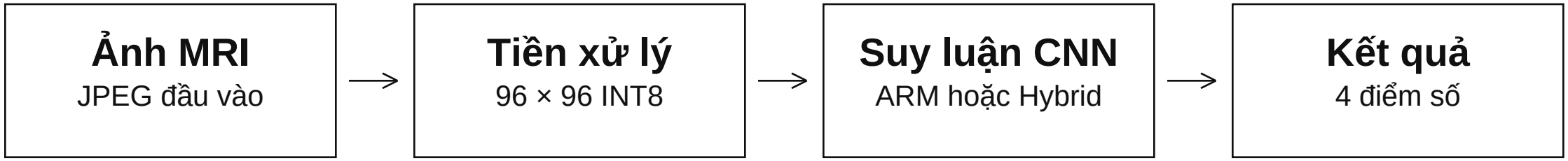
brisc2025_test_00023_gl_ax_t1.jpg

Nhãn thực: **glioma** • Nhãn dự đoán: **glioma**

Bốn lớp mô hình: glioma – meningioma – no_tumor – pituitary.

Đây là ảnh dùng trong phép thử một ảnh ở Lab 05. Không sử dụng kết quả này để suy ra độ chính xác của toàn tập.

Nhìn trước đường đi của dữ liệu

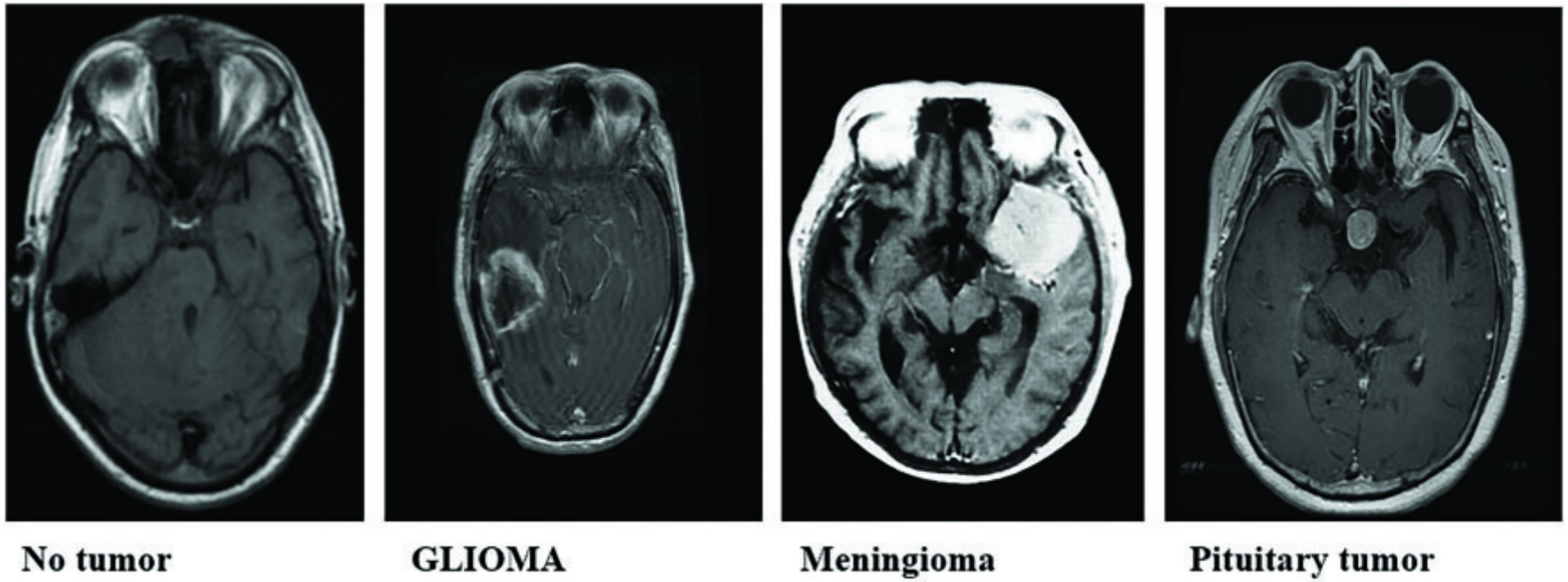


Mỗi chặng phải thống nhất với mô hình đã được huấn luyện và xuất từ Edge Impulse.

Phần đầu bài giảng đi theo một ảnh; phần cuối dùng 995 ảnh để kiểm chứng độ chính xác và hiệu năng.

Dữ liệu vào của Tiny CNN V5

Từ JPEG, chương trình tạo tensor đầu vào **96 × 96 × 1**.



Hình MRI lấy từ bộ hình minh họa đã dùng ở Slide 01, không phải chính ảnh có tên tệp của phép thử Lab 05.

Tiền xử lý ảnh: từ JPEG đến mảng số

Lab 05 sử dụng ảnh xám và nội suy bilinear để đưa về đúng kích thước.

```
img = Image.open(IMAGE).convert("L")
img = img.resize((96, 96), Image.BILINEAR)
x = np.asarray(img, dtype=np.int16) - 128
x = x.astype(np.int8).reshape(1, 96, 96, 1)
```

– **convert("L")**: ảnh chuyển về một kênh độ sáng.

– **resize**: điều chỉnh về 96 × 96 pixel.

– **reshape**: thêm chiều batch và chiều kênh cho Interpreter.

Vì sao phải trừ 128?

Pixel mức xám ban đầu là số nguyên từ 0 đến 255.

$$q = p - 128$$

Pixel ban đầu p	Sau lượng tử hóa q
0	−128
128	0
255	127

Đây là phép chuyển đổi đã dùng trong Notebook. Không mặc định áp dụng cách này cho mọi mô hình INT8.

Các kích thước tensor cần nhớ

Giai đoạn	Kích thước tensor	Dung lượng INT8
Ảnh đầu vào	96 × 96 × 1	9.216 byte
Conv1 + ReLU	94 × 94 × 8	70.688 byte
MaxPool1	47 × 47 × 8	17.672 byte
Đầu ra phân loại	1 × 4	4 phần tử INT8

Conv1 dùng kernel 3 × 3, padding valid và stride 1; MaxPool1 dùng cửa sổ 2 × 2.

Tại sao Conv1 còn 94 × 94?

Không thêm đệm (padding = valid), bộ lọc 3 × 3, bước trượt bằng 1.

$$H_{out} = 96 - 3 + 1 = 94$$

Tương tự, chiều rộng đầu ra là 94. Với **8 bộ lọc**, ta có:

$$94 \times 94 \times 8$$

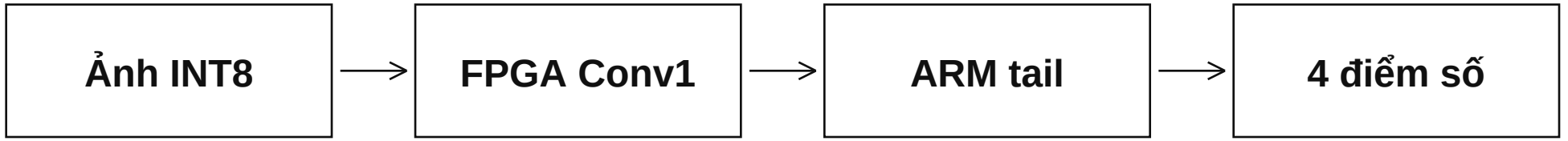
Khi áp dụng MaxPooling2D (2 × 2), mỗi chiều 94 giảm còn 47; số kênh vẫn là 8.

Hai cách suy luận cần so sánh

ARM-only



ARM + FPGA



Overlay: đưa thiết kế phần cứng lên board

Mô hình đã huấn luyện và IP tăng tốc cần các tệp khác nhau.

Tệp	Vai trò
conv1_accel.bit	Cấu hình phần cứng PL
conv1_accel.hwh	Mô tả giao tiếp phần cứng cho PYNQ
tiny_cnn_v5_int8.tflite	Mô hình CNN đầy đủ trên ARM
tiny_cnn_v5_tail_int8.tflite	Mô hình chạy sau MaxPool1

Cặp .bit/.hwh được tạo trong Lab 04; hai mô hình TFLite được dùng khi chạy Lab 05.

Khởi tạo phần cứng và hai Interpreter

```
ol = Overlay("conv1_accel.bit")

dma = ol.axi_dma_0

conv1 = ol.conv1_hls_0

full = Interpreter(model_path="tiny_cnn_v5_int8.tflite", num_threads=1)

tail = Interpreter(model_path="tiny_cnn_v5_tail_int8.tflite", num_threads=1)

full.allocate_tensors()

tail.allocate_tensors()
```

– Overlay tạo giao diện điều khiển IP và DMA từ Python.

– Interpreter thực thi mô hình TFLite trên ARM.

Bộ đệm DMA: tại sao phải cấp phát riêng?

```
ibuf = allocate(shape=(9216,), dtype=np.int8)

obuf = allocate(shape=(47, 47, 8), dtype=np.int8)
```

– **ibuf** chứa ảnh trước khi DMA truyền vào bộ tăng tốc.

– **obuf** nhận tensor đầu ra sau MaxPool1 từ FPGA.

– Bộ đệm do PYNQ cấp phát để phục vụ giao tiếp giữa DDR và phần cứng.

DDR ↔ DMA ↔ IP Conv1: dữ liệu đi đâu?



MM2S: đọc ibuf từ DDR và truyền tới IP.

S2MM: nhận dữ liệu từ IP và ghi về obuf trong DDR.

AXI DMA truyền dữ liệu; ARM vẫn chịu trách nhiệm ra lệnh và kiểm tra trạng thái.

Đẩy ảnh vào FPGA: bốn thao tác chính

```
ibuf[:] = x.reshape(-1)

ibuf.flush()

dma.recvchannel.transfer(obuf)

conv1.write(0x00, 1)

dma.sendchannel.transfer(ibuf)
```

– Đưa ảnh INT8 vào bộ đệm đầu vào và đồng bộ cache.

– Chuẩn bị nhận dữ liệu đầu ra, khởi động IP, rồi truyền ảnh vào FPGA.

Khi DMA đang chạy, Python làm gì?

– ARM kiểm tra thanh ghi trạng thái của hai hướng truyền DMA.

– Nếu có lỗi, chương trình dừng và báo rõ trạng thái TX/RX.

– Khi cả hai hướng hoàn tất, bộ đệm kết quả được **invalidate** để ARM đọc dữ liệu mới.

– Có giới hạn thời gian chờ để tránh vòng lặp vô hạn khi phần cứng không phản hồi.

ARM tiếp nhận tensor MaxPool1

```
obuf.invalidate()

pool1 = obuf.reshape(1, 47, 47, 8)

tail.set_tensor(ti, pool1)

tail.invoke()

result = tail.get_tensor(to)
```

Đây là ranh giới xử lý: FPGA đã hoàn thành Conv1 + ReLU + Pool1; ARM chỉ tiếp tục các lớp còn lại.

Một ảnh MRI được phân loại ra sao?

Kết quả thực tế trong Lab 05:

[127, −128, −128, −128]

Chỉ số lớp có điểm số lớn nhất: **0** → **glioma**.

Điểm số sau giải lượng tử hóa của lớp dự đoán xấp xỉ **0,9961**.

0,9961 là điểm số trên một ảnh; không phải Accuracy của toàn bộ ảnh Test hoặc mức chắc chắn y khoa.

Vì sao phải so sánh toàn bộ tensor?

```
if not np.array_equal(a, b):

    raise RuntimeError("Output mismatch")
```

– Nhãn dự đoán có thể giống nhau dù một số điểm số đầu ra đã khác.

– Khi **np.array_equal** trả về True, từng phần tử INT8 đầu ra khớp chính xác.

– Đây là phép kiểm chứng tính tương đương giữa đường ARM-only và ARM + FPGA.

Hai phép đánh giá cần phân biệt

Phép thử	Đối tượng	Mục đích
Lab 05	1 ảnh × 30 lượt	Kiểm chứng chức năng và độ trễ lặp
Lab 06	995 ảnh Test	Đo Accuracy, F1, thời gian toàn tập

– Không dùng điểm số 0,9961 của một ảnh để kết luận hệ thống đạt Accuracy 99,61%.

Hai thí nghiệm có phạm vi khác nhau nên số trung vị có thể khác nhau.

Đo thời gian trên một ảnh như thế nào?

– **Warm-up 5 lượt** trước khi ghi thời gian chính thức.

– **Đo 30 lượt** và luân phiên thứ tự chạy ARM / Hybrid.

– Dùng **time.perf_counter()** để lấy thời gian và tính **trung vị**.

– Thời gian bao gồm suy luận cùng chi phí DMA của Hybrid, **không gồm** đọc ảnh, resize, nạp Overlay hay khởi tạo Interpreter.

Kết quả một ảnh ở Lab 05

Thành phần	Trung vị 30 lượt
ARM-only	23,626 ms
FPGA + DMA	1,507 ms
ARM tail	16,554 ms
ARM + FPGA	18,055 ms
Hệ số tăng tốc	1,309×

Các trung vị thành phần được tính trên những dãy thời gian riêng; không nhất thiết cộng đúng bằng trung vị tổng.

Minh chứng từ Notebook thực tế ở Lab 05

Log ghi nhận một ảnh MRI chạy cả hai đường suy luận trên board.

– **ARM output** và **Hybrid output** đều là [127, −128, −128, −128].

– Chương trình in **All outputs match: True**.

– Kết quả benchmark được tính bằng trung vị của **30 lượt đo**.

– Thông báo cuối: **PASS: Hybrid CNN verified**.

```
Warming up...
Benchmarking 30 runs...

=== MRI CLASSIFICATION ===
Image: brisc2025_test_00023_gl_ax_t1.j
Predicted class index: 0
Scores: [0.9961 0.     0.     0.    ]
ARM output: [ 127 -128 -128 -128]
Hybrid output: [ 127 -128 -128 -128]
All outputs match: True

=== PERFORMANCE (MEDIAN) ===
Measured runs: 30
ARM only        : 23.626 ms
FPGA + DMA      : 1.507 ms
ARM tail        : 16.554 ms
ARM + FPGA      : 18.055 ms
Speedup         : 1.309x

PASS: Hybrid CNN verified
```

Mở rộng kiểm thử đến 995 ảnh ở Lab 06

Chỉ số	Kết quả thực tế
Ảnh phân loại đúng	922 / 995
Accuracy	92,66%
Macro F1	0,9194
Tensor INT8 ARM / Hybrid khớp	995 / 995
ARM-only / ARM + FPGA	23,644 / 17,855 ms
Speedup	1,324×

Đây mới là phép đánh giá phân loại trên toàn bộ tập Test; không suy ra từ một ảnh của Lab 05.

Diễn tập lời giảng: theo dõi một ảnh

“Đầu tiên, nhóm đọc ảnh MRI, chuyển về ảnh xám 96 × 96 và biểu diễn mỗi pixel bằng số nguyên INT8. Ở phương án ARM thuần, toàn bộ CNN chạy bằng TFLite. Trong phương án kết hợp, dữ liệu đi qua DMA đến khối Conv1 trên FPGA. Sau ReLU và MaxPooling, tensor 47 × 47 × 8 được trả về bộ nhớ và đưa vào phần mạng còn lại trên ARM.”

“Nhóm kiểm tra đầu ra INT8 của hai đường thực thi phải giống nhau, rồi mới so sánh thời gian suy luận. Đây là cách đánh giá cả tính đúng đắn và hiệu năng của giải pháp.”

Kiểm tra hiểu bài trước khi quay video

– Vì sao ảnh đầu vào có 9.216 byte nhưng đầu ra Pool1 có 17.672 byte?

– Nếu DMA đã hoàn tất, tại sao vẫn cần thao tác **invalidate()**?

– Vì sao hai mô hình TFLite **full** và **tail** không thể thay thế tùy ý cho nhau?

– Nếu nhãn dự đoán giống nhau, phép **np.array_equal()** còn có ý nghĩa gì?

– Vì sao kết quả một ảnh ở Lab 05 không phải độ chính xác chung của đề tài?

KẾT THÚC SLIDE 10

Một ảnh MRI → tiền xử lý → INT8 → DMA / FPGA → ARM → nhãn

Hãy tự giảng lại đường đi này mà không nhìn code.

Nguồn nội dung kỹ thuật: Lab 03, Lab 04, Lab 05 và Lab 06 của đề tài.